

**Министерство науки и высшего образования Российской Федерации  
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ  
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ  
«Национальный исследовательский университет ИТМО»**

**(Университет ИТМО)**

**Факультет инфокоммуникационных технологий**

**Образовательная программа Мобильные и сетевые технологии**

**Направление подготовки 09.03.03 Прикладная информатика**

**О Т Ч Е Т**

**по выполнению курсовой работы**

**По дисциплине: “Мобильная разработка (Android и iOS)”**

**Выполнил:**

**Гнеушев Владислав Алексеевич**

**Группа: К3439**

**Проверил:**

**Шатинский Григорий Сергеевич**

**Санкт-Петербург, 2026**

## СОДЕРЖАНИЕ

|                                                   |           |
|---------------------------------------------------|-----------|
| <b>ВВЕДЕНИЕ.....</b>                              | <b>3</b>  |
| <b>1 Проектирование приложения.....</b>           | <b>5</b>  |
| 1.1 Архитектурные компоненты.....                 | 5         |
| 1.2 Архитектурный паттерн MVVM.....               | 5         |
| 1.3 Используемые технологии и инструменты.....    | 6         |
| 1.4 Структура приложения.....                     | 7         |
| 1.5 Проектирование логики работы с данными.....   | 7         |
| <b>2 Реализация приложения.....</b>               | <b>8</b>  |
| 2.1 Реализация навигации и структуры экранов..... | 8         |
| 2.2 Реализация архитектуры MVVM.....              | 13        |
| 2.3 Работа с сетью и получение данных.....        | 16        |
| 2.4 Реализация локального хранилища.....          | 16        |
| 2.5 Отображение списка сообщений.....             | 17        |
| 2.6 Фоновая синхронизация и уведомления.....      | 17        |
| <b>ЗАКЛЮЧЕНИЕ.....</b>                            | <b>20</b> |

## ВВЕДЕНИЕ

Мобильные приложения являются неотъемлемой частью современных информационных систем и широко используются для коммуникации, получения информации и организации повседневной деятельности. Платформа Android занимает одно из ведущих мест на рынке мобильных устройств, что обуславливает актуальность изучения принципов разработки Android-приложений, современных архитектурных подходов и инструментов работы с данными.

В рамках данной курсовой работы рассматривается разработка Android-приложения типа «Мессенджер», реализованного с использованием архитектурных компонентов Android и паттерна MVVM. Приложение объединяет в себе навигацию на основе Activity и Fragment, управление состоянием через ViewModel и LiveData, загрузку данных из сети, локальное хранение информации в базе данных Room, а также фоновую синхронизацию и систему уведомлений.

Целью курсовой работы является разработка Android-приложения «Мессенджер», демонстрирующего применение современных технологий и архитектурных решений Android для построения масштабируемого, устойчивого к изменениям конфигурации и расширяемого мобильного приложения.

Для достижения поставленной цели в работе необходимо решить следующие задачи:

- 1) Проанализировать основные компоненты Android-приложений и принципы архитектуры MVVM.
- 2) Разработать структуру приложения с использованием Activity, Fragment и Navigation Component.
- 3) Реализовать экраны новостной ленты, профиля и настроек с использованием Bottom Navigation.

- 4) Внедрить ViewModel и LiveData для хранения и управления состоянием приложения.
- 5) Реализовать загрузку сообщений из внешнего API с использованием Retrofit и корутин.
- 6) Организовать локальное хранение данных с помощью базы Room и обеспечить офлайн-доступ.
- 7) Улучшить пользовательский интерфейс с применением компонентов Material Design.
- 8) Реализовать фоновую синхронизацию через WorkManager и систему уведомлений.

# **1 Проектирование приложения**

## **1.1 Архитектурные компоненты**

При разработке современных Android-приложений особое внимание уделяется использованию архитектурных компонентов Android Jetpack. К ним относятся Activity, Fragment, ViewModel, LiveData, Room, WorkManager и Navigation Component. Данные компоненты предоставляют набор готовых инструментов для построения масштабируемой и устойчивой архитектуры приложения.

Использование архитектурных компонентов позволяет реализовать принцип разделения ответственности между уровнями приложения, снизить связность кода и упростить сопровождение и расширение проекта. Кроме того, они обеспечивают корректную работу с жизненным циклом компонентов Android, что уменьшает риск утечек памяти и ошибок, связанных с пересозданием экранов.

## **1.2 Архитектурный паттерн MVVM**

В данной курсовой работе в качестве основы архитектуры выбран паттерн MVVM (Model–View–ViewModel).

Архитектура MVVM предполагает разделение приложения на три основных уровня:

- Model — слой данных, включающий репозитории, работу с сетью, базой данных и бизнес-логику.
- View — пользовательский интерфейс (Activity и Fragment), отвечающий за отображение данных и обработку действий пользователя.
- ViewModel — промежуточный слой, который хранит состояние приложения, подготавливает данные для отображения и обеспечивает связь между Model и View.

Ключевым преимуществом MVVM является отсутствие прямой зависимости между пользовательским интерфейсом и источниками данных. ViewModel не содержит ссылок на Activity или Fragment, что делает ее

устойчивой к пересозданию экранов и позволяет сохранять данные при повороте устройства.

Для реактивного обновления интерфейса в архитектуре MVVM используются компоненты LiveData. Они позволяют автоматически уведомлять пользовательский интерфейс об изменениях данных и обеспечивают безопасное взаимодействие с жизненным циклом Android-компонентов.

### **1.3 Используемые технологии и инструменты**

В ходе разработки приложения были использованы следующие основные технологии:

- 1) Activity и Fragment — базовые компоненты пользовательского интерфейса.
- 2) Navigation Component — для организации навигации между экранами и управления back stack.
- 3) ViewModel и LiveData — для хранения состояния приложения и реактивного обновления UI.
- 4) Retrofit — для сетевого взаимодействия с внешним API.
- 5) Kotlin Coroutines — для выполнения асинхронных операций без блокировки главного потока.
- 6) Room — для организации локальной базы данных и офлайн-доступа.
- 7) RecyclerView — для отображения списка сообщений.
- 8) Material Design Components — для улучшения пользовательского интерфейса.
- 9) WorkManager — для фоновой синхронизации данных.
- 10) Notification API — для уведомления пользователя о новых данных.

Использование данного набора технологий соответствует современным стандартам Android-разработки и обеспечивает корректную работу приложения в условиях нестабильного сетевого соединения и изменений жизненного цикла.

## **1.4 Структура приложения**

Проектируемое приложение представляет собой упрощенный мессенджер с тремя основными экранами: лента сообщений, профиль пользователя и настройки.

В качестве точки входа используется одна главная Activity, внутри которой размещен NavHostFragment. Навигация между экранами реализована при помощи Bottom Navigation и Navigation Graph. Каждый экран представлен отдельным Fragment, что обеспечивает модульность интерфейса и упрощает расширение функциональности.

## **1.5 Проектирование логики работы с данными**

Для централизованной работы с данными был реализован репозиторий сообщений, который является единой точкой взаимодействия ViewModel с источниками информации. Репозиторий получает данные из сетевого API и сохраняет их в локальной базе данных Room.

При наличии интернет-соединения выполняется загрузка данных из сети и обновление базы. При отсутствии сети данные извлекаются из локального хранилища, что обеспечивает офлайн-доступ к сообщениям.

Дополнительно была предусмотрена возможность периодического обновления данных с помощью WorkManager.

## 2 Реализация приложения

### 2.1 Реализация навигации и структуры экранов

Приложение построено по одноактивитийному принципу: одна главная Activity выступает контейнером, а экраны реализованы фрагментами (см. листинг 1). Это упрощает структуру проекта, потому что Activity занимается инфраструктурой (тема, разрешения, навигация), а логика и UI отдельных разделов вынесены в самостоятельные фрагменты.

#### Листинг 1 – MainActivity

```
class MainActivity : AppCompatActivity() {
    private lateinit var binding: ActivityMainBinding
    private val permissionLauncher = registerForActivityResult(
        ActivityResultContracts.RequestMultiplePermissions()
    ) { }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        Log.d("MainActivity", "onCreate")

        ThemeManager.applyTheme(this)

        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)

        requestPermissionsIfNeeded()

        try {
            val navHostFragment = supportFragmentManager
                .findFragmentById(R.id.nav_host_fragment) as
NavHostFragment
            val navController = navHostFragment.navController

            binding.bottomNavigation.setupWithNavController(navController)
```



```

        Log.d("MainActivity", "Navigation setup successful")
    } catch (e: Exception) {
        Log.e("MainActivity", "Navigation setup failed", e)
    }
}

override fun onStart() {
    super.onStart()
    Log.d("MainActivity", "onStart")
}

override fun onResume() {
    super.onResume()
    Log.d("MainActivity", "onResume")
}

override fun onPause() {
    super.onPause()
    Log.d("MainActivity", "onPause")
}

override fun onStop() {
    super.onStop()
    Log.d("MainActivity", "onStop")
}

override fun onDestroy() {
    super.onDestroy()
    Log.d("MainActivity", "onDestroy")
}

private fun requestPermissionsIfNeeded() {
    val permissions =
mutableListOf(Manifest.permission.READ_CONTACTS)

```

```

        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU)
        {

permissions.add(Manifest.permission.POST_NOTIFICATIONS)

        }
        val needRequest = permissions.any { permission ->
            ContextCompat.checkSelfPermission(this, permission) !=
android.content.pm.PackageManager.PERMISSION_GRANTED
        }
        if (needRequest) {
            permissionLauncher.launch(permissions.toTypedArray())
        }
    }
}

```

Навигация реализована через Jetpack Navigation. В главном layout размещен NavHostFragment, который управляет отображением экранов по графу навигации. Нижнее меню выполнено через Bottom Navigation и связано с контроллером навигации, поэтому переключение вкладок происходит автоматически, без ручных транзакций фрагментов.

Граф навигации содержит три экрана: лента сообщений, профиль и настроек (экраны показаны на рисунках 1-3). Такой набор отражает основную структуру приложения и делает переходы предсказуемыми для пользователя. Для демонстрации жизненного цикла добавлено логирование ключевых методов Activity и Fragment, что позволяет отследить, когда создаются экраны, когда их view пересоздается и как это связано с навигацией или поворотом экрана.



Рисунок 1 – Экран “Новости”

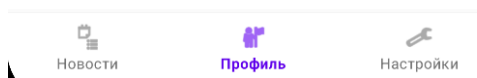
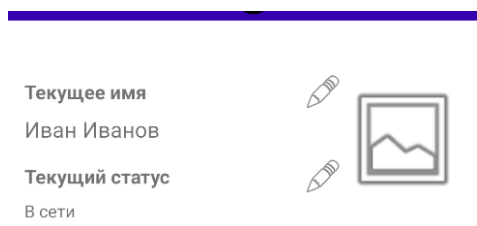


Рисунок 2 – Экран “Профиль”

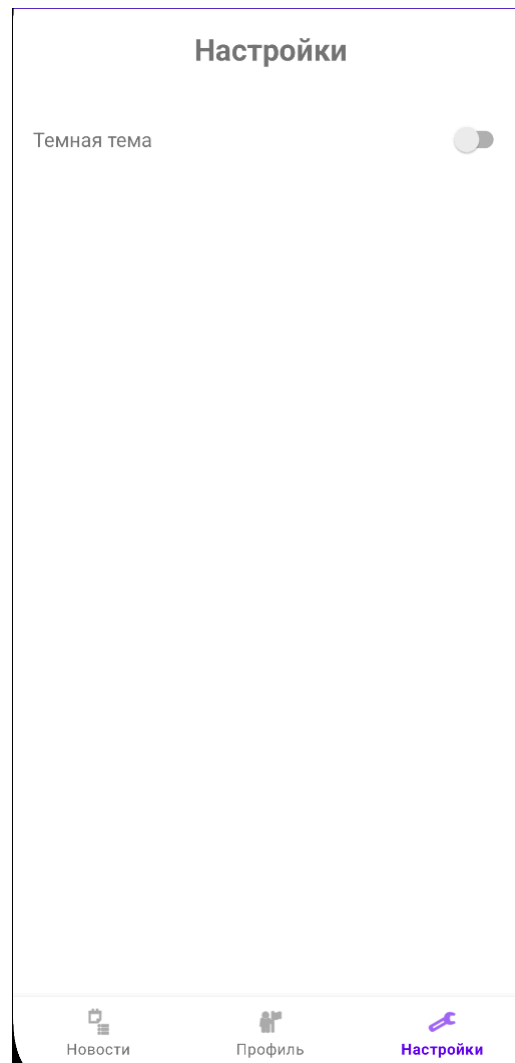


Рисунок 3 – Экран “Настройки”

## 2.2 Реализация архитектуры MVVM

В проекте используется MVVM: фрагменты отображают состояние и отправляют события, а ViewModel хранит данные и выполняет действия (см. листинг 2). За счет этого состояние не привязано к жизненному циклу view и корректно переживает пересоздание экранов при повороте.

### Листинг 2 – Пример ViewModel

```
class NewsViewModel(application: Application) :  
    AndroidViewModel(application) {  
    private val repository: MessageRepository =  
        MessageServiceLocator.provideRepository(application)
```

```

        private val _uiState = MutableStateFlow(FeedUiState(isLoading
= false))
        val uiState: StateFlow<FeedUiState> = _uiState.asStateFlow()

        init {
            observeMessages()
            refreshMessages()
        }

        private fun observeMessages() {
            viewModelScope.launch {
                repository.messages.collect { messages ->
                    _uiState.update { it.copy(messages = messages) }
                }
            }
        }

        fun refreshMessages() {
            viewModelScope.launch {
                _uiState.update { it.copy(isLoading = true,
errorMessage = null) }
                val result = repository.refreshMessages()
                val fallbackError =
getApplication<Application>().getString(R.string.news_error_defaul
t)

                val errorText = if (result.isFailure) {
                    result.exceptionOrNull()?.localizedMessage?.takeIf
{ it.isNotBlank() }?.let { msg ->
                        "Не удалось обновить данные: $msg"
                    } ?: fallbackError
                } else {
                    null
                }
                _uiState.update {
                    it.copy(

```

```

        isLoading = false,
        isOffline = result.isFailure,
        errorMessage = errorText
    )
}
}
}

fun toggleLike(messageId: Int) {
    _uiState.update { state ->
        val newSet = state.likedIds.toMutableSet()
        if (newSet.contains(messageId)) {
            newSet.remove(messageId)
        } else {
            newSet.add(messageId)
        }
        state.copy(likedIds = newSet)
    }
}
}

```

Для профиля и настроек выделена общая ViewModel, которая хранит имя, статус и выбранную тему, а UI подписывается на LiveData и обновляет элементы интерфейса только при изменениях. Общий экземпляр модели используется сразу в двух экранах, поэтому профиль и настройки синхронизированы без дополнительных связей.

Экран ленты использует корутины и StateFlow: ViewModel хранит единое состояние (список сообщений, загрузка, офлайн-режим, ошибки и локальные лайки), а фрагмент собирает поток только когда находится в активном состоянии жизненного цикла. Это обеспечивает безопасную связь UI и логики и исключает лишнюю работу в фоне.

## **2.3 Работа с сетью и получение данных**

Данные для ленты загружаются по сети через Retrofit. Запрос описан в отдельном API-интерфейсе как suspend-метод, поэтому вызов выполняется в корутине и не блокирует главный поток. В качестве источника используется публичный тестовый сервис, что удобно для демонстрации сетевого слоя и формата ответов.

Создание клиента и зависимостей (Retrofit, OkHttp, конвертер JSON) вынесено в отдельный модуль и переиспользуется во всем приложении. Сетевой вызов выполняется на фоне, а результат передается наверх в виде успеха или ошибки, чтобы UI мог одинаково обработать оба сценария.

Ошибки обрабатываются централизованно: при проблемах с сетью экран показывает текст ошибки и признак офлайн-режима, но остается работоспособным за счет локального кэша. Для ручного обновления предусмотрена кнопка «Обновить», которая запускает загрузку, отображает индикатор прогресса и временно блокируется, чтобы не создавать параллельные запросы.

## **2.4 Реализация локального хранилища**

Локальное хранилище реализовано на базе Room. Для сообщений описана сущность таблицы с первичным ключом, а доступ к данным вынесен в DAO. Чтение выполняется через Flow, поэтому интерфейс получает обновления автоматически при изменении содержимого базы.

Обновление кэша выполнено как атомарная операция: при успешной загрузке данные полностью заменяются в таблице, чтобы локальное состояние соответствовало серверу. База создается один раз на уровне приложения, что исключает утечки и повторные инициализации.

Офлайн-режим обеспечивается тем, что лента всегда показывает данные из базы, а сеть используется только для обновления. Репозиторий выступает единой точкой доступа: ViewModel не зависит от деталей Retrofit или Room, а работает с абстракцией “получить поток сообщений” и “обновить данные”.



## 2.5 Отображение списка сообщений

Лента сообщений отображается через RecyclerView: задается менеджер компоновки, подключается адаптер и настраиваются обработчики действий пользователя. RecyclerView переиспользует элементы при прокрутке, поэтому подходит для списков любого размера и не перегружает память.

Для отображения элементов используется кастомный адаптер с ViewHolder и привязкой через ViewBinding. В момент привязки данных заполняются основные поля сообщения и выставляется нужная иконка лайка. Нажатие на лайк передается наружу в виде события, чтобы адаптер оставался “тонким” и не содержал бизнес-логики.

Состояние лайков хранится на уровне ViewModel как набор идентификаторов, поэтому UI может быстро переотрисовать нужные элементы и сохранить состояние при повороте экрана. В учебной версии лайки являются локальной функцией экрана и не синхронизируются с сервером.

## 2.6 Фоновая синхронизация и уведомления

Фоновая синхронизация реализована через WorkManager. При старте приложения планируется периодическая задача с ограничением “только при наличии сети”, поэтому система сама выбирает подходящий момент запуска и не расходует ресурсы в офлайне. Работа ставится как уникальная, чтобы не создавать несколько одинаковых задач.

Воркер выполняет обновление данных через репозиторий и возвращает успех или запрос на повтор, если произошла ошибка. При успешном обновлении показывается уведомление: для новых версий Android создается канал уведомлений, а для Android 13+ дополнительно учитывается разрешение на показ уведомлений. Реализация воркера показана в листинге 3.

Листинг 3 – Воркер, обновляющий сообщения

```
class MessageSyncWorker(  
    applicationContext: Context,  
    workerParams: WorkerParameters  
) : CoroutineWorker(applicationContext, workerParams) {
```

```

        override suspend fun doWork(): Result {
            val repository =
MessageServiceLocator.provideRepository(applicationContext)
            val result = repository.refreshMessages()
            return if (result.isSuccess) {
                showNotification()
                Result.success()
            } else {
                Result.retry()
            }
        }

private fun showNotification() {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU)
{
        val permission = ContextCompat.checkSelfPermission(
            applicationContext,
            Manifest.permission.POST_NOTIFICATIONS
        )
        if (permission != PackageManager.PERMISSION_GRANTED) {
            return
        }
    }

    val channelId = "message_sync"

            val manager =
applicationContext.getSystemService(Context.NOTIFICATION_SERVICE)
as NotificationManager

        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            val channel = NotificationChannel(
                channelId,

applicationContext.getString(R.string.notification_channel_name),
                NotificationManager.IMPORTANCE_DEFAULT

```

```

        )
        manager.createNotificationChannel(channel)
    }

    val notification =
NotificationCompat.Builder(applicationContext, channelId)
        .setSmallIcon(R.drawable.ic_notification)

        .setContentTitle(applicationContext.getString(R.string.notification_title))

        .setContentText(applicationContext.getString(R.string.notification_text))

        .setAutoCancel(true)
        .build()

    manager.notify(1, notification)
}
}

```

Таким образом, приложение реагирует на онлайн/офлайн и на уровне интерфейса, и на уровне фоновой логики: при отсутствии сети лента продолжает работать на кэше и показывает признак офлайна, а фоновая синхронизация автоматически откладывается до появления подключения.

## **ЗАКЛЮЧЕНИЕ**

В ходе выполнения курсовой работы было разработано Android-приложение «Мессенджер» с применением современного архитектурного паттерна MVVM и компонентов Android Jetpack. Приложение включает три основных экрана с навигацией через Bottom Navigation, реализует загрузку данных из сети с помощью Retrofit, обеспечивает локальное хранение через базу данных Room и поддерживает фоновую синхронизацию с использованием WorkManager.

Разработанное приложение демонстрирует применение современных подходов к построению масштабируемых мобильных приложений. Использование архитектурных компонентов обеспечило устойчивость к изменениям конфигурации, корректную работу с жизненным циклом и возможность офлайн-доступа к данным.